

Company: Google

Interview Type: CASE_STUDY

Q1: Distributed Token Bucket for Global Rate Limiting

Difficulty: Hard

Tags: Distributed Systems, Concurrency, Algorithms

Problem Statement: Design a system to enforce a global rate limit (e.g., 10,000 requests per second) for a specific API across 50 geographically distributed Google Cloud regions. The solution must minimize cross-region latency for individual requests while ensuring the global limit is not exceeded by more than 5%.

Expected Approach: Use a centralized Redis or Spanner instance to track a global counter. Each request performs a remote atomic increment. This ensures accuracy but introduces massive latency due to cross-continental round trips (e.g., 200ms+).

Optimized Approach: Implement a "Hierarchical Token Bucket" with local "Bursting." Each region is allocated a sub-quota (e.g., 200 QPS). If a region exceeds its quota, it requests a larger batch of tokens (e.g., 1000) from the global coordinator. Regions use an asynchronous background thread to sync usage and rebalance quotas based on traffic weight.

Time Complexity: $O(1)$ for local check; $O(1)$ amortized for global sync.

Space Complexity: $O(N)$ where N is the number of API keys/users being tracked.

Optimizations: Use "Token Borrowing" where under-utilized regions proactively release tokens to a global pool. Implement "Max-Min Fairness" to ensure small users aren't starved by heavy hitters during global congestion.

Hints: How can we reduce the frequency of global network calls? What happens if the global coordinator is temporarily unreachable?

Q2: Dependency-Aware Distributed Build Scheduler

Difficulty: Hard

Tags: Graph Theory, Dynamic Programming, Parallel Computing

Problem Statement: You are building "Bazel-Lite." Given a Directed Acyclic Graph (DAG) of 1 million build targets and their dependencies, determine an execution order that minimizes total build time on a cluster with K worker nodes. Each target has a known estimated execution time.

Expected Approach: Simple Topological Sort using Kahn's Algorithm. While this produces a valid order, it does not account for target execution times or worker availability, leading to suboptimal parallelization ($O(V+E)$).

Optimized Approach: Use a "Critical Path Scheduler." Calculate the "Height" of each node (longest path from the node to a leaf) using DP. Maintain a Priority Queue of ready-to-execute nodes

(in-degree 0), prioritized by their Height. At each step, assign the node with the largest Height to the next available worker.

Time Complexity: $O(V \log V + E)$ for calculating paths and priority queue management.

Space Complexity: $O(V + E)$ to store the adjacency list and path weights.

Optimizations: Implement "Worker Affinity": assign child nodes to the same worker that completed the parent node to minimize data transfer latency ($O(1)$ lookup via hashmap).

Hints: Think about which nodes are on the "Critical Path"—if they are delayed, the whole build is delayed. How do we prioritize those?

Q3: Hierarchical Global Quota Controller

Difficulty: Hard

Tags: Tree, Distributed Systems, Concurrency, Hash Map

Problem Statement: Google Cloud needs a system to manage API quotas organized in a hierarchy: Organization > Folder > Project > Service. Each node in this tree has a defined "Max Requests Per Second" (RPS). When a request arrives at a specific Service, it must not exceed the RPS limit of its own node OR any of its ancestors. Design an interface to validate requests in real-time across millions of nodes.

Expected Approach: Represent the hierarchy as a tree where each node contains an atomic counter and a limit. For every incoming request, traverse from the leaf (Service) to the root, checking and incrementing counters. Use a Mutex per node to ensure thread safety.

Optimized Approach: Store the tree in a flat Hash Map using unique Resource IDs for $O(1)$ access. Implement a "Token Bucket" algorithm at each node. To minimize lock contention in a distributed environment, use a "Lease" system where local edge servers pre-allocate a batch of tokens from the global controller. Only synchronize with the global state when the local batch is exhausted or the lease expires.

Time Complexity: $O(H)$ per request, where H is the height of the tree (typically constant/small).

Space Complexity: $O(N)$ where N is the total number of resource nodes in the hierarchy.

Optimizations: Use Read-Copy-Update (RCU) for the tree structure since the hierarchy changes less frequently than request volume. Implement "Lazy Evaluation" for ancestor counters.

Hints: How do you handle the "Thundering Herd" problem when a high-level limit is reached? Can we use SWR (Stale-While-Revalidate) for quota checks?

Q4: Real-time Proximity-based Notification Engine

Difficulty: Medium-Hard

Tags: Spatial Indexing, Quadtree/S2, Stream Processing

Problem Statement: Google Maps wants to notify drivers when they enter a "Dynamic Congestion Zone." There are 10^7 active drivers providing GPS updates every 5 seconds and 10^5 active circular congestion zones. Design a system to identify which drivers have entered a zone within 1

second of their location update.

Expected Approach: For every location update received, iterate through all 10^5 zones and calculate the Haversine distance between the driver and the zone center. If distance < radius, trigger a notification.

Optimized Approach: Use a spatial indexing system like S2 Geometry or a Quadtree. Divide the map into cells. Index all congestion zones by the cells they cover. When a driver's location update arrives, map their coordinates to a Cell ID. Only perform distance checks against the zones indexed in that specific cell and its immediate neighbors.

Time Complexity: $O(1)$ or $O(\log N)$ for cell lookup; $O(K)$ for distance checks, where K is the small number of zones in a cell.

Space Complexity: $O(Z + D)$ where Z is the number of zones and D is the number of active drivers.

Optimizations: Use a "Movement Delta" threshold; if a driver has moved less than 10 meters, skip the re-calculation. Use a Bitset to track which zones a driver is already "inside" to avoid duplicate notifications.

Hints: How do you handle zones that are larger than a single grid cell? Consider the trade-offs between fixed-size grids and hierarchical grids like S2.

Q5: Proximity-Aware Task Scheduling in Geo-Distributed Clusters

Difficulty: Hard

Tags: Greedy, Min-Priority Queue, Heuristics, Distributed Systems

Problem Statement: You are designing a scheduler for Google Cloud. There are N tasks and M data centers. Each task i has a data dependency of size S_i located at data center D_i . Each data center has K available computation slots. To run task i at data center j , the cost is calculated as: $\text{Cost} = (\text{TransferRate} \times \text{Distance}(D_i, j) \times S_i) + \text{CurrentQueueDelay}(j)$. Given a distance matrix between all data centers and a stream of tasks, design an algorithm to assign each task to a data center to minimize the global average completion time (latency).

Expected Approach: Use a Greedy approach where for each incoming task, calculate the cost across all M data centers. Assign the task to the data center that yields the minimum instantaneous cost. Update the CurrentQueueDelay for that data center by adding the task's execution time.

Optimized Approach: Implement a Min-Priority Queue per data center to track slot availability. To handle the global optimum better than pure greedy, use a "Min-Min" heuristic: for a batch of B tasks, calculate the minimum completion time for each task across all DCs, then pick the task that has the overall minimum completion time and assign it. Update and repeat.

Time Complexity: $O(B^2 * M)$ for batch scheduling or $O(N * M)$ for stream scheduling.

Space Complexity: $O(M + N)$ to store DC states and task assignments.

Optimizations: Use a K-D Tree to represent data centers geographically to prune the search space for the "closest" data center if the distance is physical.

Hints: Think about how task execution time affects future tasks assigned to the same DC. Is the closest data center always the best if it has a long queue?

Q6: Efficient Range Metadata Merging for Cloud Storage

Difficulty: Hard

Tags: Segment Tree, Lazy Propagation, Interval Merging, Data Structures

Problem Statement: Google Cloud Storage manages objects as contiguous byte ranges. Each range has a "Storage Class" metadata (e.g., COLDLINE, STANDARD). Users can perform `SetClass(start_byte, end_byte, class_type)` operations. If, after an update, two or more adjacent byte ranges have the same `class_type`, they must be merged into a single range to minimize metadata overhead. Design a system to handle U updates and return the final minimal list of metadata blocks.

Expected Approach: Use a Map or a Balanced Binary Search Tree (like `std::set` in C++ or `TreeMap` in Java) to store the start points of intervals. For every update, split the existing intervals at `start_byte` and `end_byte`, remove all intermediate intervals, and insert the new range. Then, check the immediate neighbors to merge if the `class_type` matches.

Optimized Approach: Use a Segment Tree with Lazy Propagation. Each node in the tree represents a byte range and stores the `class_type` if the entire range is uniform. When an update hits a node, if the update covers the node's range, update the "lazy" value and the node's class. During the update, if a node's children end up having the same class, propagate upward to merge.

Time Complexity: $O(U \log N)$, where N is the total byte address space (e.g., 2^{64}), but practically $O(U \log K)$ where K is the number of existing intervals.

Space Complexity: $O(K)$ to store the intervals.

Optimizations: Since the address space is 2^{64} , use a Dynamic Segment Tree (allocate nodes on demand) or a Coordinate Compression technique on the U update boundaries.

Hints: How do you handle an update that only partially overlaps with three different existing storage classes? What is the most efficient way to check if the neighbor to the left can be merged?

Q7: Global Presence Service for Real-Time Collaborative Editors

Difficulty: Hard

Tags: Distributed Systems, WebSockets, Pub/Sub, Low Latency

Problem Statement: Design a system to track and broadcast the real-time cursor positions and "active" status of millions of users across millions of shared documents (e.g., Google Docs). The system must handle 100k+ concurrent users on a single popular document while maintaining sub-100ms latency for cursor updates.

Expected Approach: Use a relational database to store user coordinates and poll every 500ms. Broadcast updates to all users via a standard REST API. This fails at scale due to database write amplification and high HTTP overhead.

Optimized Approach: Use WebSockets for persistent bi-directional communication. Shard the "Presence Servers" by Document ID. Use an in-memory Pub/Sub (like Redis or NATS) where each Document ID is a topic. Implement "Interest Management": only broadcast cursor updates of users within the current viewing frustum/viewport of other users to reduce egress bandwidth. Use ephemeral TTLs in a distributed cache to handle "last seen" heartbeats.

Time Complexity: $O(1)$ for update and broadcast to subscribers.

Space Complexity: $O(N + M)$ where N is the number of active users and M is the number of active documents.

Optimizations: Use Protocol Buffers (Binary format) to minimize payload size. Implement "Delta Updates" to only send changed coordinates. Use a "Gossip Protocol" for cluster membership of presence nodes.

Hints: How do we handle a "celebrity document" where 100k people watch one person type? Does every user need to know where everyone else is, or just those nearby?

Q8: Scalable Near-Duplicate Document Detection for Web Crawling

Difficulty: Hard

Tags: Information Retrieval, Hashing, Big Data, MapReduce

Problem Statement: Googlebot crawls billions of web pages daily. Many pages are near-duplicates (e.g., same content with different timestamps or localized ads). Design a system to determine if a newly crawled page (1MB average) is a "near-duplicate" of any previously indexed page without performing pairwise comparisons.

Expected Approach: Compute a SHA-256 hash of the entire page content. This fails because a single character change (like a timestamp) results in a completely different hash, missing near-duplicates.

Optimized Approach: Use SimHash or MinHash (Locality Sensitive Hashing). 1. Tokenize the document into n -grams. 2. Project n -grams into a fixed-length bit vector (fingerprint) where similar documents have a small Hamming distance. 3. Store fingerprints in a multi-index table (permutations) to allow sub-linear search. 4. During crawling, generate the fingerprint and query the index for existing fingerprints with a Hamming distance $\leq k$ (typically $k=3$).

Time Complexity: $O(L)$ to hash (L = doc length), $O(1)$ or $O(\log N)$ for index lookup.

Space Complexity: $O(N * F)$ where N is documents and F is fingerprint size (e.g., 64 bits).

Optimizations: Use a distributed MapReduce job to batch-process fingerprints. Implement "Weighting" (TF-IDF) on tokens before hashing to ignore boilerplate text like "Privacy Policy."

Hints: Think about how to represent a document as a set of features. How can we make the hash "stable" even if the input changes slightly?

Q9: Hierarchical Global Rate Limiter for Google Cloud

Difficulty: Hard

Tags: Distributed Systems, Token Bucket, Hierarchical Partitioning, Consistency

Problem Statement: Google Cloud needs to enforce API rate limits across 50+ global regions. A specific enterprise user has a global limit of 100,000 Requests Per Second (RPS). If one region (e.g., us-east) experiences a traffic spike while another (e.g., asia-south) is idle, the system must dynamically reallocate the unused quota to the saturated region without hitting a single central coordinator for every request.

Expected Approach: Use a centralized Redis cluster to track global counts. Each request performs a network round-trip to the global counter. This causes massive latency and a single point of failure.

Optimized Approach: Implement a Hierarchical Token Bucket. Each regional cluster is granted a "lease" of the global quota (e.g., 2,000 RPS). Local nodes handle requests using a local lock-free counter. When a region's local quota drops below a threshold (e.g., 20%), it asynchronously requests a larger lease from the Global Quota Manager. If a region is under-utilizing, it returns its lease.

Time Complexity: $O(1)$ for local request validation; $O(N)$ periodic sync where N is the number of regions.

Space Complexity: $O(U)$ where U is the number of active global users.

Optimizations: Use "Predictive Borrowing" based on the derivative of the traffic growth curve to request leases before the local bucket empties.

Hints: How do you handle a total network partition between a region and the global manager? Should you "fail-open" or "fail-closed"?

Q10: Real-time Dynamic Route Rerouting for Autonomous Delivery

Difficulty: Hard

Tags: Graph Theory, Contraction Hierarchies, Spatial Indexing, A* Search

Problem Statement: A fleet of 10,000 delivery robots operates in a city represented as a graph of 1M nodes. Road conditions (obstructions, human traffic) change every 5 seconds, updating edge weights. Design a system that provides the shortest path for all robots, ensuring that an edge weight update only triggers a re-calculation for robots whose current paths are actually affected.

Expected Approach: Periodically re-run Dijkstra's algorithm for all robots every time an edge weight changes. This results in $O(R * (E + V \log V))$ complexity, which is computationally infeasible for real-time updates.

Optimized Approach: Utilize Customizable Route Planning (CRP) with S2 Geometry cells. Partition the city into hierarchical cells. Precompute an "overlay graph" that contains only the distances between boundary nodes of these cells. When an edge weight changes, only the overlay distances for that specific cell need re-computation. Use a "Path Corridor" check: if the modified edge is within the bounding box of a robot's current path, only then trigger a re-route.

Time Complexity: $O(\log V)$ for local cell updates; $O(E')$ for overlay updates where $E' \ll E$.

Space Complexity: $O(V + E)$ for the base graph + $O(P)$ for precomputed boundary shortcuts.

Optimizations: Use bitmask-based pruning to quickly skip cells that are geographically opposite to the robot's destination.

Hints: Consider how many cells a typical path intersects. Is it necessary to update the entire overlay immediately?
